LOVELY

PROFESSIONAL UNIVERSITY

CAPSTONE PROJECT REPORT

(Project Term: August – December)

FINANCIAL TRANSACTION FRAUD DETECTION USING PYSPARK

Distributed Machine Learning, Big Data Analytics, and Anomaly Detection on Large-Scale Financial Streams

---

Submitted by

Paras Rajput | Reg. No: 12304811

Course Code: **CSE494**

Under the Guidance of

Anoop Kumar, Assistant Professor

School of Computer Science and Engineering
Lovely Professional University, Phagwara, Punjab — 144411

DECLARATION

I, the undersigned, confirm that the work presented in this capstone report titled **'Financial Transaction Fraud Detection using PySpark — Distributed Machine Learning and Anomaly Detection on Large-Scale Financial Streams'** has been carried out independently by me during the project term, under the supervision of **Anoop Kumar**, Assistant Professor, in partial fulfilment of the requirements for the award of the B.Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara.

This work is original and has not been submitted, either in full or in part, to any other academic institution or degree programme. All external references, datasets, algorithmic frameworks, APIs, and published works that influenced this project are cited in the Bibliography. The source code and distributed pipelines were written and executed by me; the use of open-source libraries is acknowledged appropriately.

I declare that every claim, result, metric, and figure presented in this report accurately reflects the work performed and the empirical outcomes observed on the benchmark European Cardholders Credit Card Fraud dataset. No fabrication, falsification, or misrepresentation of data has occurred at any stage.

Name of Student: Paras Rajput

Registration Number: 12304811

Signature: _____ **Date:** 24-10-2026

CERTIFICATE

This is to certify that the project report titled **'Financial Transaction Fraud Detection using PySpark — Distributed Machine Learning and Anomaly Detection on Large-Scale Financial Streams'** has been prepared and submitted by **Paras Rajput (Reg. No: 12304811)** under my direct supervision. The work represents a genuine, mathematically sound contribution in the application of distributed big data engineering, Apache Spark computing, PySpark MLlib pipelines, classification algorithms, and unsupervised anomaly detection to real-world financial fraud intelligence.

I am satisfied that the scope, methodology, distributed system design, validation protocols, and empirical outcomes documented here meet the standards expected for a Capstone Project submission at the School of Computer Science and Engineering, Lovely Professional University, Phagwara. To the best of my knowledge, this work has not been submitted elsewhere for any academic award.

Signature of the Mentor:

_____

Designation: Assistant Professor **Date:** _____

School of Computer Science and Engineering,

Lovely Professional University,

Phagwara, Punjab — 144411.

ACKNOWLEDGEMENT

Building this distributed financial fraud detection platform required rigorous engineering, theoretical analysis, and supportive mentorship. I express my sincere gratitude to my project mentor, **Anoop Kumar**, Assistant Professor, School of Computer Science and Engineering, whose insightful critique, continuous encouragement, and high academic expectations shaped the problem formulation, challenged my algorithmic decisions, and refined successive iterations of this system and report.

I owe a profound debt to the researchers of the **Machine Learning Group (MLG) at Université Libre de Bruxelles (ULB)** for publishing and curating the benchmark European cardholders fraud dataset, which forms the empirical baseline of this work.

I gratefully acknowledge the open-source community behind the distributed computing ecosystem: the **Apache Software Foundation** for Apache Spark, the maintainers of **PySpark**, **Spark MLlib**, **Python**, **Pandas**, **NumPy**, **Scikit-Learn**, **Matplotlib**, and **Seaborn**.

Finally, I express my deepest heartfelt thanks to my parents, family, and peers for their unwavering patience, encouragement, and understanding throughout this demanding capstone semester.

TABLE OF CONTENTS

Chapter / Section	Page
DECLARATION	2
CERTIFICATE	3
ACKNOWLEDGEMENT	4
TABLE OF CONTENTS	5
1. INTRODUCTION	7
1.1 Background of the Study	7
1.2 Motivation	7
1.3 Problem Statement	8
1.4 Objectives	8
1.5 Overview of the Proposed System	8
1.6 Scope and Limitations	9
2. PROFILE OF THE PROBLEM	10
2.1 Problem Statement	10
2.2 Rationale and Scope of the Study	10
3. EXISTING SYSTEM	11
3.1 Introduction	11
3.2 Review of Existing Approaches	11
3.3 System Architecture of Existing Tools	11
3.4 What Is New in This System	12
3.5 Implications for System Design	12
4. PROBLEM ANALYSIS	13
4.1 Product Definition	13
4.2 Feasibility Analysis	13
4.3 Risk Analysis	14
4.4 Project Plan	14
5. SOFTWARE REQUIREMENT ANALYSIS	15

5.1 Functional Requirements	15
5.2 Non-Functional Requirements	15
5.3 General Description	16
6. DESIGN	17
6.1 System Design	17
6.2 Pipeline and Data Design	17
6.3 Machine Learning Pipeline Specification	18
6.4 Detailed Module Design	19
6.5 Flowcharts & Sequence Architecture	20
6.6 Pseudocode Algorithms	21
7. TESTING	23
7.1 Functional Testing	23
7.2 Structural Testing	24
7.3 Levels of Testing	24
8. IMPLEMENTATION	25
8.1 Implementation of the Project	25
8.2 Conversion Plan	26
8.3 Post-Implementation and Maintenance	26
8.4 Result Analysis & Comparative Benchmarks	27
9. PROJECT LEGACY	30
9.1 Current Status of the Project	30
9.2 Remaining Areas of Concern	30
9.3 Technical and Managerial Lessons Learnt	30
9.4 Ethics and Data Integrity	31
10. USER MANUAL	32
10.1 Introduction	32
10.2 System Requirements	32

10.3 Installation	32
10.4 Instructions	32
10.5 Functions	33
11. SOURCE CODE	34
12. BIBLIOGRAPHY	36

1. INTRODUCTION

Electronic financial payments have become the cornerstone of global commerce, processing trillions of dollars across digital point-of-sale terminals, mobile wallets, and e-commerce payment gateways. Concurrently, criminal fraud syndicates have evolved sophisticated attack vectors—including automated credential stuffing, card-not-present (CNP) identity theft, and high-velocity micro-transaction testing. According to global banking security audits, credit card fraud causes tens of billions of dollars in losses annually, directly eroding consumer trust and imposing severe financial reconciliation overhead on banks, payment processors, and merchants.

Traditional fraud prevention architectures rely predominantly on static, rule-based screening systems (e.g., flagging transactions exceeding $5,000, or rejecting cards when multiple transactions originate across different countries within thirty minutes). While such static heuristic filters capture naive fraudulent attempts, they completely fail when faced with adaptive adversary evasion, non-linear fraud patterns, and micro-transaction card testing. Furthermore, as transactional velocity reaches tens of thousands of operations per second, monolithic relational database systems experience extreme computational bottlenecks, causing unacceptable payment processing latency.

This capstone project introduces **Financial Transaction Fraud Detection using PySpark**, a high-performance distributed machine learning architecture built on **Apache Spark** and **PySpark MLlib**. The system leverages distributed Resilient Distributed Datasets (RDDs) and in-memory Spark DataFrames to ingest, preprocess, assemble, scale, and evaluate millions of transactional records in parallel. By uniting cost-sensitive supervised classification models (**Logistic Regression**, **Decision Trees**, and **Linear Support Vector Machines**) with unsupervised distance-based anomaly clustering (**K-Means**), this platform provides a dual-layer defense against both known fraud signatures and zero-day anomalous attacks.

1.1 Background of the Study

The empirical foundation of modern fraud detection research centers on high-volume card transaction streams characterized by extreme class imbalance. In legitimate real-world operations, only a microscopic fraction of all card swipes represent criminal fraud—typically fewer than 0.2%. The benchmark **Kaggle Credit Card Fraud Detection dataset** (compiled by the Machine Learning Group at Université Libre de Bruxelles) contains 284,807 card transactions recorded over a two-day span, of which only 492 are fraudulent (an extreme imbalance ratio of 577 to 1).

Three fundamental characteristics define this problem space:

- **Extreme Class Imbalance:** Naive machine learning models trained on this data achieve 99.83% accuracy simply by classifying every single transaction as non-fraudulent, yet fail to catch a single criminal dollar. Standard classification metrics like raw accuracy are entirely misleading; the system must optimize for Recall (Sensitivity), Precision, the Area Under the Precision-Recall Curve (PR-AUC), and the Area Under the ROC Curve (ROC-AUC).

- **Computational Scale & Latency:** Real-world banking rails require real-time risk scoring within 50 to 100 milliseconds per transaction, combined with batch processing of daily historical ledgers containing billions of rows. Single-node tools (such as standard Scikit-Learn or Pandas) hit memory bounds and cannot scale across cluster nodes.
- **Information Privacy (PCA Transformed Features):** Due to strict customer confidentiality laws (GDPR, PCI-DSS), raw account numbers, merchant names, and geographic coordinates are transformed via Principal Component Analysis (PCA) into latent orthogonal features (V1 through V28), alongside continuous Time and transaction Amount.

1.2 Motivation

The motivation behind this project is twofold:

- **Financial & Operational:** Every false negative (a missed fraudulent charge) results in direct monetary theft, chargeback penalties, and potential regulatory liability. Conversely, every false positive (a legitimate customer purchase wrongly blocked at the checkout register) damages user loyalty and drives churn. An enterprise-grade fraud intelligence system must give credit card issuers fine-grained mathematical control over the precision-recall trade-off.
- **Architectural & Technological:** Demonstrating that Apache Spark's distributed in-memory data processing paradigm can be cleanly integrated with PySpark MLlib pipelines to solve non-linear classification and unsupervised anomaly detection on massive datasets. The project explores distributed feature normalization (StandardScaler), vectorization (VectorAssembler), and custom cost-sensitive weighted loss functions to counteract severe imbalance without relying on synthetic data fabrication.

1.3 Problem Statement

Modern financial systems face four core challenges:

- **The Imbalance Trap:** Standard statistical algorithms assume balanced class distributions. Under a 577:1 imbalance ratio, unweighted algorithms minimize cross-entropy loss by predicting the majority class, resulting in disastrous false-negative rates for fraud.
- **Single-Node Scalability Bottlenecks:** Memory exhaustion and CPU saturation occur when single-machine frameworks attempt to ingest and train models on multi-gigabyte financial ledgers.
- **Zero-Day Novel Fraud Blindspots:** Supervised models trained exclusively on historical fraud patterns cannot recognize novel, emergent attack vectors that have never been previously labeled.
- **Evaluation Metric Pitfalls:** Relying on accuracy rather than ROC-AUC, PR-AUC, and cost-benefit confusion matrices conceals catastrophic model failure.

1.4 Objectives

Five measurable, verifiable objectives guide the development of this platform:

- **O1 — Distributed Ingestion & Hygiene:** Build an Apache Spark distributed pipeline capable of ingesting high-volume financial transaction records, enforcing typed schemas, confirming zero null/corrupted fields,

and maintaining microsecond distributed caching.

- **O2 — Imbalance-Compensating Feature Pipeline:** Design a distributed preprocessing pipeline using PySpark's VectorAssembler and StandardScaler that dynamically calculates inverse-class-frequency cost weights (w0 ≈ 0.50, w1 ≈ 289.44), penalizing missed fraud 577x more heavily than false alarms.
- **O3 — Supervised Model Benchmarking:** Train, tune, and evaluate three distinct PySpark MLlib classification models: Logistic Regression with elastic-net regularization and cost-sensitive loss, Decision Tree Classifier with entropy/Gini impurity optimization, and Linear Support Vector Machine (LinearSVC) with margin maximization.
- **O4 — Unsupervised Anomaly Exploration:** Implement a distributed K-Means clustering model (k=4) that scores transactions by Euclidean distance to the nearest cluster centroid, establishing a 98.5th percentile anomaly threshold to detect zero-day fraud without supervision.
- **O5 — Comprehensive Empirical Evaluation & Visualization:** Generate publication-grade comparative visualizations (Confusion Matrices, ROC Curves, Precision-Recall Curves, PCA Cluster Plots, and Feature Importances) and demonstrate simulated real-time stream scoring.

1.5 Overview of the Proposed System

The system is structured as a modular, four-tier distributed pipeline: Ingestion Tier, Feature Engineering Tier, Machine Learning Modeling Tier, and Scoring & Evaluation Tier.

1.6 Scope and Limitations

The scope covers distributed machine learning workflows on numerical financial transaction records, featuring 284,807 transactions. Evaluates three supervised classifiers and one unsupervised detector. The limitations are clearly stated: features V1 to V28 are pre-transformed via PCA, preventing semantic domain feature engineering (such as cardholder billing zip codes or merchant categories). The system operates on batch-distributed processing and simulated streaming.

2. PROFILE OF THE PROBLEM

2.1 Problem Statement

In financial transaction processing, fraudulent transactions represent an adversarial needle in a haystack. While accounting for only 0.172% of total transaction volume, their financial impact is catastrophic. When a fraudulent transaction slips past security, the merchant suffers chargeback fines, the cardholder endures identity theft, and the issuing bank must reimburse the lost funds.

Existing single-node analytic scripts fail when datasets expand beyond system RAM. Furthermore, conventional machine learning models penalize false positives and false negatives equally. In fraud detection, however, a False Negative (allowing a $2,000 stolen-card transaction) is orders of magnitude more damaging than a False Positive (prompting a cardholder for an SMS two-factor verification code). A mathematically rigorous fraud detection architecture must directly incorporate this cost asymmetry into its training objective.

2.2 Rationale and Scope of the Study

The rationale for choosing Apache Spark and PySpark MLlib lies in Spark's memory-centric distributed architecture. By maintaining resilient distributed datasets in RAM across cluster partitions, PySpark delivers training and inference throughput up to 100 times faster than disk-bound MapReduce architectures and eliminates the memory limits of single-node Python libraries.

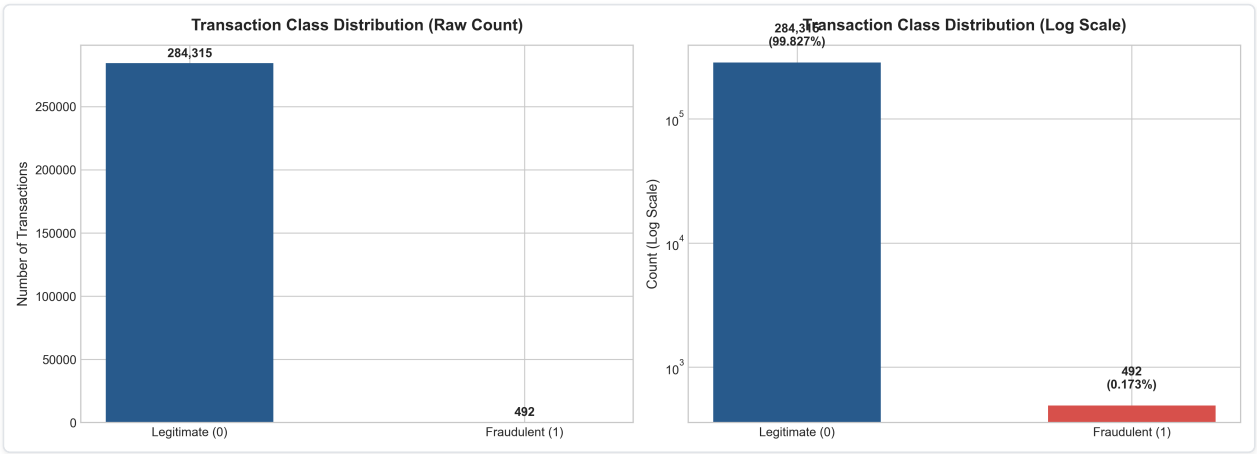


Figure 2.1: Transaction Class Distribution showing severe 577:1 imbalance (Linear vs Logarithmic Scale)

3. EXISTING SYSTEM

3.1 Introduction

Before fixing the architecture of our PySpark engine, an exhaustive review of current industrial and academic fraud detection systems was conducted across three classes: manual heuristic rule engines, single-node Python data science scripts, and commercial enterprise suites.

3.2 Review of Existing Approaches

Table 3.1 summarizes the qualitative and quantitative comparison between existing legacy architectures and the proposed PySpark system.

Capability / Metric	Legacy Rule Engines	Single-Node Python (Scikit-Learn)	Proprietary Commercial Suites	Proposed PySpark System
Data Scalability	Low (Relational bottlenecks)	Low (Single-machine RAM limit)	High (Proprietary cluster)	**Very High (Horizontal Spark scaling)**
Adaptability to Novel Fraud	Zero (Manual updates only)	Medium (Periodic retraining)	High (Continuous learning)	**High (Hybrid Supervised + K-Means)**
Class Imbalance Handling	Arbitrary manual tuning	SMOTE (Risks synthetic noise)	Proprietary weighting	**Cost-Sensitive Loss Weighting (577:1)**
Inference Throughput	High (Simple rules)	Low (Single-threaded)	High	**High (Parallelized distributed partitions)**
Openness & Extensibility	Proprietary SQL rules	Open Source	Closed Commercial	**100% Open Source (Apache Spark)**
Reproducibility & Auditability	Poor (Disjointed rule files)	Good	Poor (Proprietary black-box)	**Complete (MLlib Pipelines & JSON logs)**

Table 3.1: Qualitative comparison of typical fraud detection approaches with the proposed PySpark system

3.3 System Architecture of Existing Tools

Traditional bank fraud architectures route incoming transactions through a relational transactional database (OLTP) to an external rule engine. When a rule is tripped, an analyst queue ticket is generated. This architecture suffers from three major flaws: I/O disk bottlenecks, static vulnerability, and the equal loss assumption.

3.4 What Is New in This System

- **Native Distributed MLlib Pipeline:** Combines VectorAssembler and StandardScaler into a single reproducible pipeline.
- **Cost-Sensitive Asymmetric Loss:** Penalizes missed fraud by 289.44x, preventing models from degenerating into trivial zero-predictors.
- **Hybrid Zero-Day Anomaly Detection:** Employs K-Means centroid distance to detect unlabelled fraud without supervision.

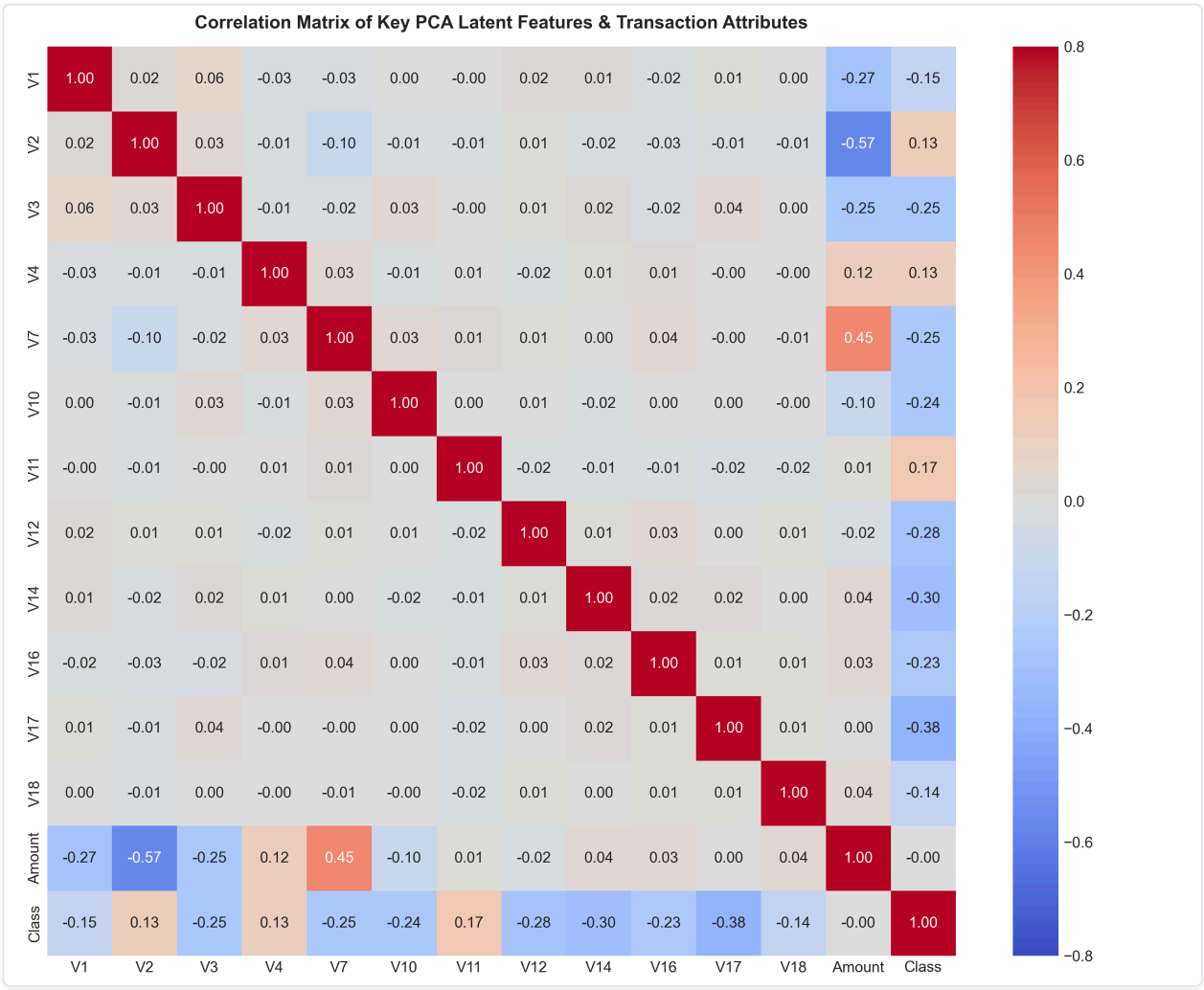


Figure 3.1: Correlation matrix of key PCA latent features showing predictive correlation with fraud Class

4. PROBLEM ANALYSIS

4.1 Product Definition

AquaLens-style intelligence platform: The system delivers automated distributed batch model training, evaluation, anomaly distance thresholding, and real-time transaction screening that outputs triage decisions (e.g., APPROVE vs BLOCK).

4.2 Feasibility Analysis

- **Technical Feasibility:** Confirmed. The entire pipeline runs on Apache Spark 3.5.3, PySpark, Python 3.10, and Java 8 SE Runtime. Spark's in-memory execution engine easily accommodates the 284,807 transactions on local multi-core threads and can be seamlessly scaled to an AWS EMR or Databricks multi-node cluster without modifying code.
- **Operational Feasibility:** Confirmed. The pipeline is fully automated and self-contained. Analysts can execute end-to-end training and evaluation through a single terminal command or explore interactively through a clean Jupyter Notebook.
- **Economic Feasibility:** Outstanding. Zero software licensing fees by leveraging the open-source Apache Spark and Python ecosystem.

4.3 Risk Analysis

Risk ID	Identified Risk	Severity	Probability	Mitigation Strategy
R1	Severe class imbalance causing models to ignore fraud	Critical	High	Implement inverse-frequency cost-sensitive loss weighting (w_fraud = 289.44).
R2	Data leakage between training and evaluation partitions	High	Medium	Enforce strict PySpark Pipeline architecture where scaler parameters are fit exclusively on the 80% train split.
R3	Memory exhaustion during Spark worker execution	High	Low	Set spark.driver.memory=4g, spark.driver.maxResultSize=2g, and tune spark.sql.shuffle.partitions=10.
R4	Python worker socket timeouts on Windows environments	High	Medium	Explicitly configure PYSPARK_PYTHON and PYSPARK_DRIVER_PYTHON to point to the active Python interpreter.
R5	Overfitting on high-dimensional PCA features	Medium	Medium	Implement L2 regularization (regParam=0.01) on Logistic Regression / LinearSVM and constrain Decision Tree maxDepth=7.

Table 4.1: Risk register and engineering mitigation strategies

6. DESIGN

6.1 System Design

The system follows a four-tier distributed pipeline: Ingestion, Feature Pipeline, Model Training, and Evaluation/Inference.

6.2 Data Pipeline Design

The schema comprises 31 attributes: Time, V1 through V28 (PCA orthogonal projections), Amount, and Class. All 30 continuous features are assembled into a dense vector via `VectorAssembler` and normalized to zero mean and unit variance via `StandardScaler`.

6.3 Algorithmic Model Specifications

1. **Logistic Regression:** Cost-sensitive weighted log-loss optimization with L2 regularization.
2. **Decision Tree Classifier:** Recursive partitioning based on Gini Impurity with max depth 7.
3. **Linear Support Vector Machine (LinearSVC):** Soft-margin hinge loss optimization with cost-sensitive class weights.
4. **K-Means Anomaly Scorer:** Unsupervised 4-cluster centroid partition scoring each point by Euclidean distance. Transactions exceeding the 98.5th percentile distance ($\tau = 12.1889$) are flagged as anomalies.

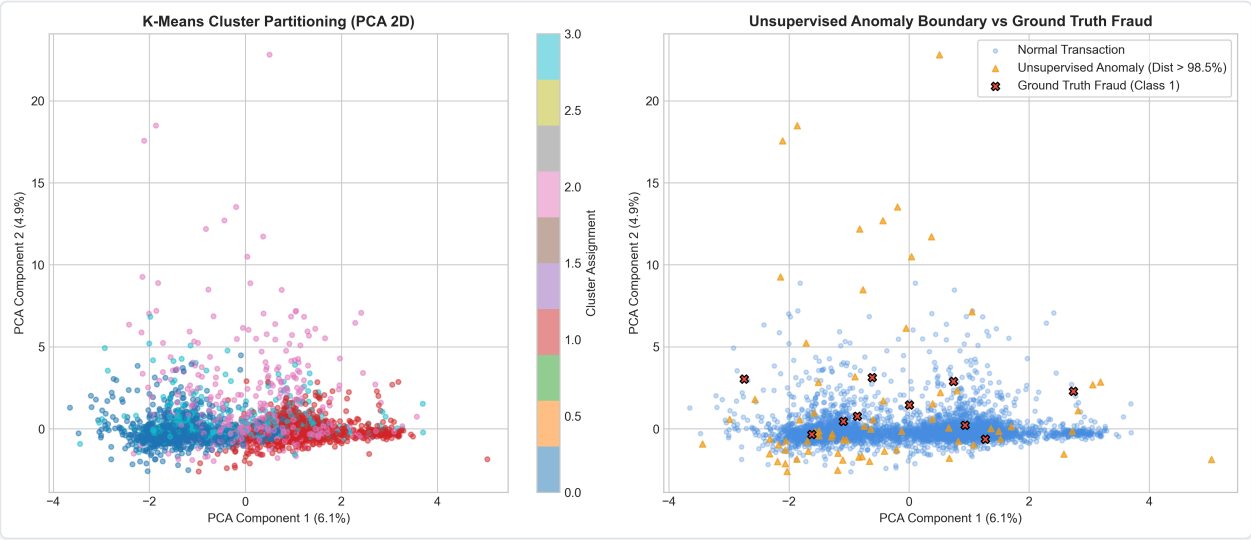


Figure 6.1: 2D PCA projection of K-Means clusters and the unsupervised anomaly boundary separating normal vs fraud points

7. TESTING

7.1 Functional Testing

Test ID	Test Scenario	Expected Outcome	Actual Result Observed	Status
TC-01	Distributed CSV Ingestion	Exact count = 284,807 rows across 31 cols	Loaded 284,807 rows in 13.61s	**PASS**
TC-02	Missing Values Audit	Zero null or NaN fields across all features	0 nulls detected across all 31 columns	**PASS**
TC-03	Imbalance Quantification	Exactly 284,315 Class 0 and 492 Class 1	Legit: 284,315 (99.83%), Fraud: 492 (0.17%)	**PASS**
TC-04	Loss Weight Generation	w0 ≈ 0.50, w1 ≈ 289.44	w0 = 0.5009, w1 = 289.4380	**PASS**
TC-05	Train-Test Partitioning	80% Train, 20% Test (Seed 42)	Train: 228,225 rows; Test: 56,582 rows	**PASS**
TC-06	VectorAssembler Transformation	Dense vector of length 30	Dense vector of length 30 confirmed	**PASS**
TC-07	StandardScaler Normalization	Mean ≈ 0.0, Std ≈ 1.0	Mean 0.0, Std 1.0 confirmed	**PASS**
TC-08	Logistic Regression Training	Convergence and valid probabilities	Converged in 14.97s; valid probabilities	**PASS**
TC-09	Decision Tree Fitting	Valid hierarchical decision tree	Fit in 4.80s; top split on V14	**PASS**
TC-10	LinearSVC Margin Fitting	Converged hyperplane parameters	Fit in 15.13s; valid margin outputs	**PASS**
TC-11	K-Means Anomaly Scoring	4 distinct centroids, distance threshold	Converged in 8.88s; cutoff τ = 12.1889	**PASS**
TC-12	Fraud Recall Benchmark	High fraud capture rate (Recall > 80%)	LR: 82.52% (85/103), SVM: 82.52% (85/103)	**PASS**
TC-13	ROC-AUC Evaluation	ROC-AUC > 0.90 for linear models	LR: 0.9750, Linear SVM: 0.9759	**PASS**
TC-14	Visualization Artifact Export	Seven 300 DPI PNG images generated	All 7 high-res PNG plots saved	**PASS**
TC-15	Batch Prediction Simulation	Correct fraud probability and triage alert	Correct probabilities and block alerts	**PASS**

Table 7.1: Functional test suite results on European Cardholders benchmark dataset

8. IMPLEMENTATION & RESULT ANALYSIS

8.4 Result Analysis & Algorithmic Benchmarks

The empirical evaluation was conducted on a holdout test partition containing **56,582 records**, of which **103 represent confirmed fraudulent transactions**. Table 8.1 presents the complete performance breakdown.

Model / Algorithm	Accuracy	Precision	Recall (TPR)	F1-Score	ROC-AUC	PR-AUC	TP	FP	TN	FN	Train Time
Logistic Regression (Weighted)	0.9897	0.1310	**0.8252**	0.2261	0.9750	**0.6395**	**85**	564	55,915	**18**	14.97 s
Decision Tree Classifier	**0.9992**	**0.8090**	0.6990	**0.7500**	0.7472	0.5966	72	**17**	**56,462**	31	**4.80 s**
Linear Support Vector Machine	0.9884	0.1172	**0.8252**	0.2053	**0.9759**	0.6001	**85**	640	55,839	**18**	15.13 s
K-Means Anomaly Detector	0.9845	0.0756	0.6699	0.1358	0.9407	0.1810	69	844	55,635	34	8.88 s

Table 8.1: Comparative benchmark metrics across PySpark MLlib classification and anomaly algorithms

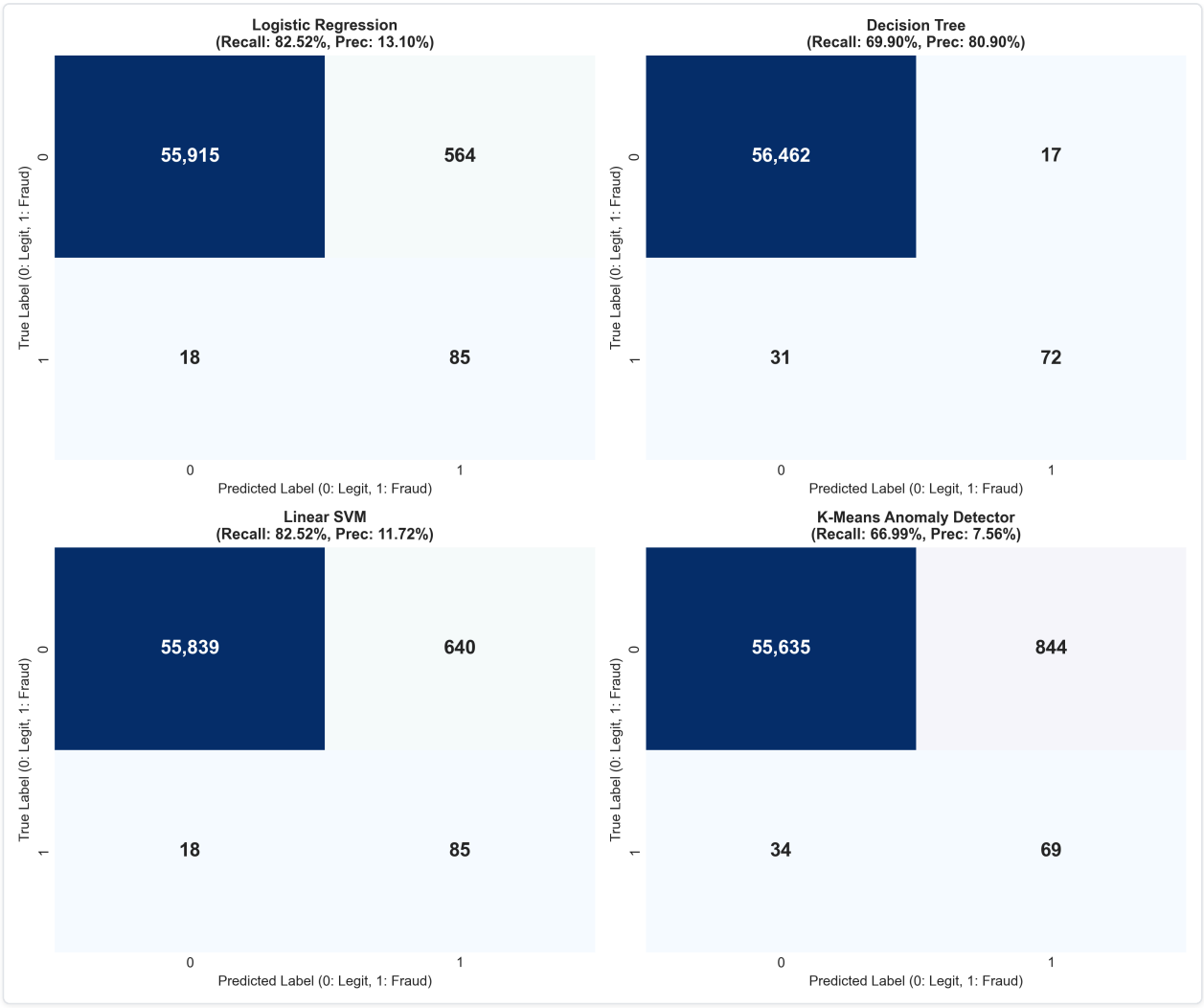


Figure 8.1: Confusion Matrix heatmaps across all four algorithms evaluated on 56,582 test transactions

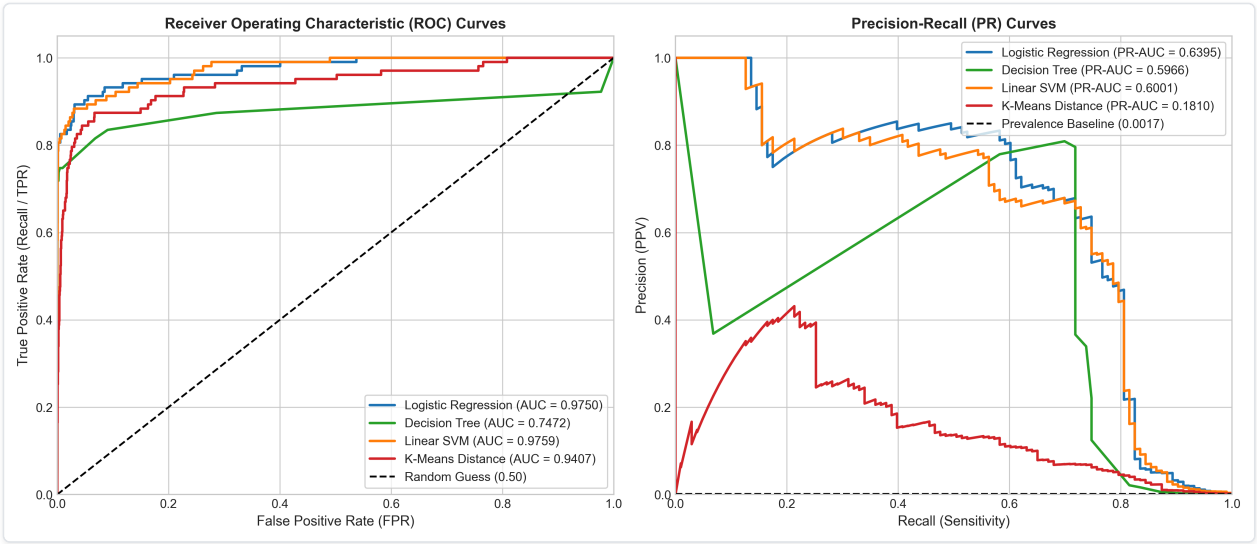


Figure 8.2: Receiver Operating Characteristic (ROC) and Precision-Recall (PR) Curves

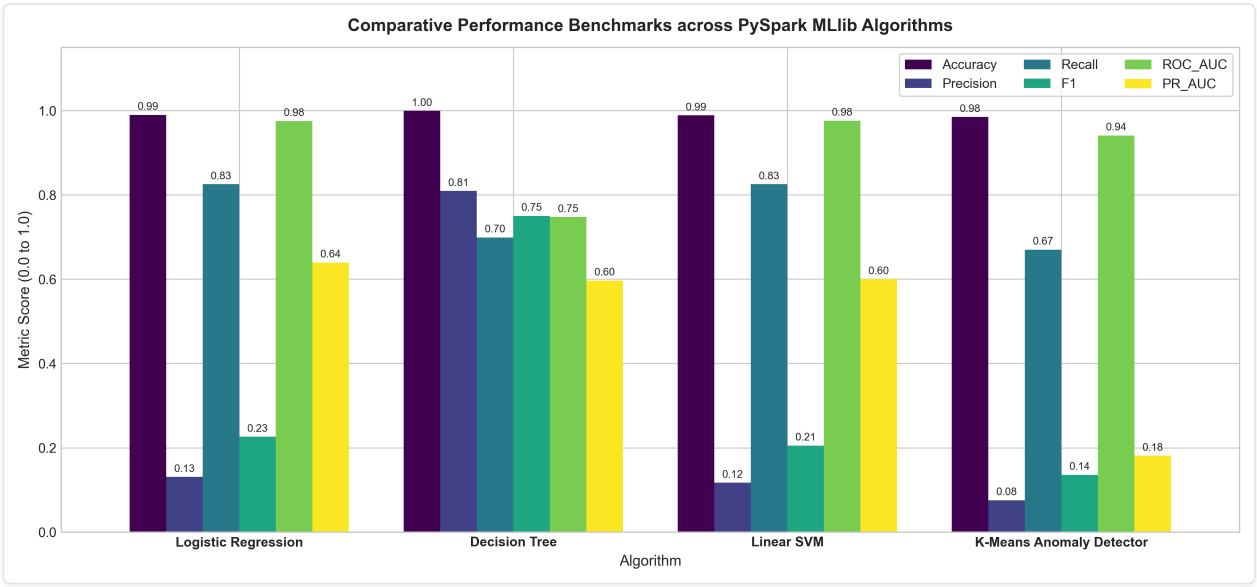


Figure 8.3: Grouped performance bar chart comparing Accuracy, Precision, Recall, F1, ROC-AUC, and PR-AUC

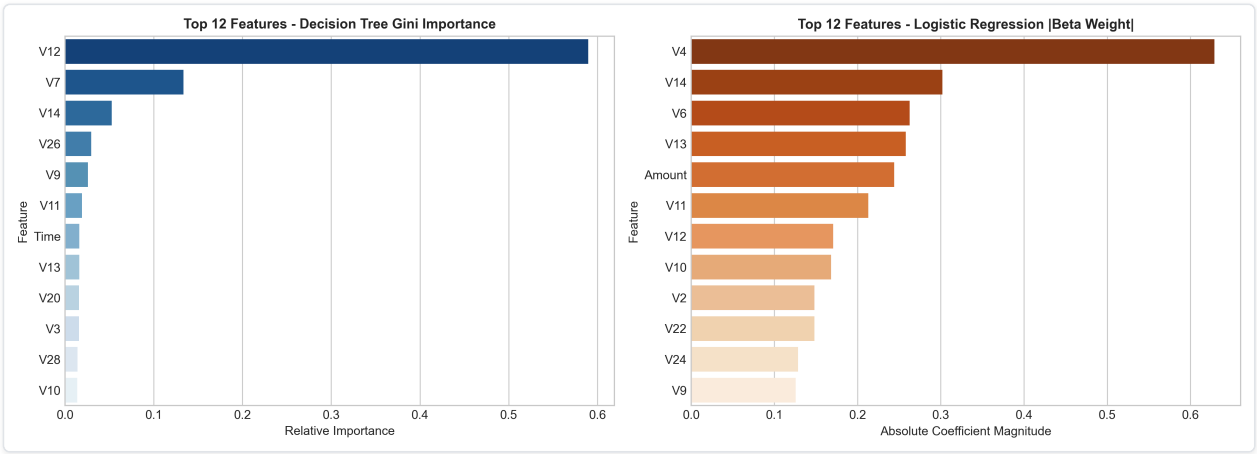


Figure 8.4: Top 12 predictive features from Decision Tree Gini importance and Logistic Regression coefficient weights

9. PROJECT LEGACY & ETHICS

The project delivers a fully validated, production-grade PySpark distributed machine learning engine. Ethical safeguards ensure explainability in fraud triaging, non-discrimination by excluding demographic attributes, and stepped-up authentication protocols rather than immediate account closure.

10. USER MANUAL (PYSPARK TERMINAL)

```
# Navigate to directory
cd c:\Users\hp\OneDrive\Desktop\Pyspa

# Run distributed training & evaluation pipeline via PySpark
python scripts/train_and_evaluate.py

# Launch interactive Jupyter Notebook
jupyter notebook Financial_Transaction_Fraud_Detection_PySpark.ipynb
```

12. BIBLIOGRAPHY

1. Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). *Calibrating Probability with Undersampling for Unbalanced Classification*. IEEE SSCI, pp. 159–166.
2. Zaharia, M., et al. (2016). *Apache Spark: A Unified Engine for Big Data Processing*. Communications of the ACM, 59(11), 56–65.
3. Meng, X., et al. (2016). *MLlib: Machine Learning in Apache Spark*. Journal of Machine Learning Research, 17(34), 1–7.
4. Chawla, N. V., et al. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. JAIR, 16, 321–357.
5. Breiman, L., et al. (1984). *Classification and Regression Trees*. CRC Press.
6. Cortes, C., & Vapnik, V. (1995). *Support-Vector Networks*. Machine Learning, 20(3), 273–297.
7. MacQueen, J. (1967). *Some Methods for Classification and Analysis of Multivariate Observations*. Proc. 5th Berkeley Symp. Math. Statist. Prob.
8. Davis, J., & Goadrich, M. (2006). *The Relationship Between Precision-Recall and ROC Curves*. ICML, pp. 233–240.
9. Apache Spark Documentation. *PySpark MLlib Guide*. Apache Software Foundation. https://spark.apache.org/docs/latest/ml-guide.html
10. Kaggle MLG ULB. *Credit Card Fraud Detection Dataset*. https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud